

NS-3 Port — Implementation Plan

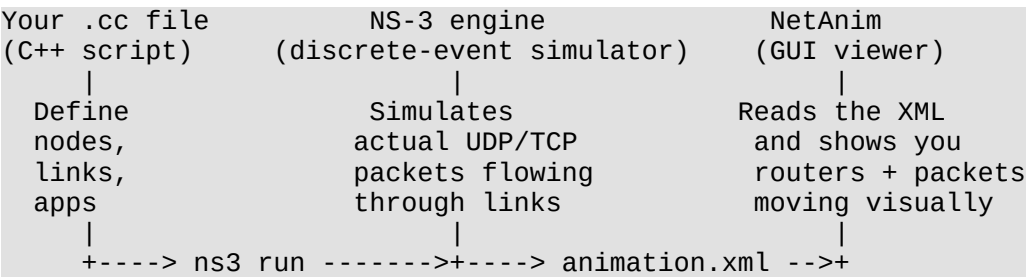
What Is This About?

We built a **terminal-based** C++ routing simulator (the code in `Network Simulator/`). Now we're porting it to **NS-3** — the industry-standard network simulator — so you get:

1. **Graphical visualization** (NetAnim shows routers as dots, packets flying between them)
2. **Realistic network simulation** (actual IP packets, queuing delays, bandwidth limits)
3. **Research credibility** (NS-3 is used in published papers)

[!IMPORTANT] The NS-3 code will live in its **own directory** inside `ns-3.48/scratch/`. It will NOT touch our existing C++ simulator.

How NS-3 Works (Quick Explanation)



In simple terms:

- You write a C++ script that says "create 100 routers, connect them, send packets"
- NS-3's engine simulates it with realistic timing (propagation delay, queuing, etc.)
- It outputs an `animation.xml` file
- NetAnim reads that file and shows a **graphical animation** of packets flowing

Your Setup (Already Working)

Component	Status	Location
NS-3 v3.48	Built	/mnt/c/Users/shrir/Downloads/ns-3.48
NetAnim	Built	~/netanim/build/netanim
WSLg GUI	Working	Kali Linux WSL2
Required modules	All present	point-to-point, internet, flow-monitor, netanim, applications, mobility

Proposed Changes

[NEW] Simulation Directory

We'll create a **subdirectory** inside NS-3's scratch folder:

```
ns-3.48/scratch/
└─ routing-sim/                # Our new directory
   └─ routing-sim.cc           # Main NS-3 simulation script
      └─ CMakeLists.txt        # Tell NS-3 to build it
```

[!NOTE] NS-3's scratch system supports subdirectories. We just need a `CMakeLists.txt` that registers our `.cc` file.

[NEW] `scratch/routing-sim/routing-sim.cc`

This is the **main simulation file** (~300 lines). Here's what it does:

Step 1: Command-Line Configuration (like our `config.txt`)

```
./ns3 run "routing-sim --routers=100 --topology=random --latency=5ms --
packets=50"
```

All parameters configurable via NS-3's built-in `CommandLine` parser:

Parameter	Default	Description
<code>--routers</code>	20	Number of routers
<code>--topology</code>	random	random, ring, mesh, tree
<code>--dataRate</code>	10Mbps	Link bandwidth
<code>--linkDelay</code>	5ms	Per-link propagation delay
<code>--packets</code>	100	Number of UDP packets to send
<code>--packetSize</code>	512	Bytes per packet
<code>--source</code>	0	Source node ID
<code>--destination</code>	-1	Dest node (-1 = last)

Step 2: Create Nodes

```
NodeContainer nodes;
nodes.Create(numRouters); // e.g., 100 NS-3 node objects
```

Step 3: Build Topology (same 4 types as our simulator)

For each link, use NS-3's `PointToPointHelper`:

```

PointToPointHelper p2p;
p2p.SetDeviceAttribute("DataRate", StringValue("10Mbps"));
p2p.SetChannelAttribute("Delay", StringValue("5ms"));
// Install on each pair of connected nodes

```

- **Random:** spanning tree + extra edges (same algorithm as ours)
- **Ring:** node $i \leftrightarrow \text{node } (i+1) \% N$
- **Mesh:** every pair connected
- **Tree:** binary tree (node $i \rightarrow 2i+1, 2i+2$)

Step 4: Install Internet Stack + Routing

```

InternetStackHelper internet;
internet.Install(nodes);
// NS-3 has built-in global routing (similar to our BFS):
Ipv4GlobalRoutingHelper::PopulateRoutingTables();

```

[!NOTE] NS-3's `GlobalRoutingHelper` does the same thing as our BFS — it computes shortest-path routing tables for all nodes. The key difference: NS-3 uses actual IP addresses and real IP forwarding.

Step 5: Assign IP Addresses

Each point-to-point link gets its own /30 subnet:

```

Ipv4AddressHelper address;
address.SetBase("10.1.1.0", "255.255.255.252");
// Assign incrementally for each link

```

Step 6: Send Packets (UDP)

```

// UDP echo server on destination
UdpEchoServerHelper server(9);

// UDP echo client on source → sends N packets
UdpEchoClientHelper client(destAddr, 9);
client.SetAttribute("MaxPackets", UIntegerValue(numPackets));
client.SetAttribute("PacketSize", UIntegerValue(packetSize));

```

Step 7: NetAnim Visualization

```

AnimationInterface anim("animation.xml");
// Position nodes in a circle/grid layout for nice visualization
// Color-code source (green) and destination (red)
anim.UpdateNodeColor(source, 0, 255, 0);
anim.UpdateNodeColor(dest, 255, 0, 0);

```

Step 8: FlowMonitor Statistics

```

FlowMonitorHelper flowHelper;
Ptr<FlowMonitor> monitor = flowHelper.InstallAll();
// After simulation: print per-flow throughput, delay, packet loss

```

[NEW] scratch/routing-sim/CMakeLists.txt

```
build_lib_example(  
  NAME routing-sim  
  SOURCE_FILES routing-sim.cc  
  LIBRARIES_TO_LINK  
    ${libpoint-to-point}  
    ${libinternet}  
    ${libapplications}  
    ${libnetanim}  
    ${libflow-monitor}  
    ${libmobility}  
)
```

What You'll See

After running the simulation:

1. **Terminal output:** Per-flow stats (throughput, avg delay, packet loss)
2. **animation.xml:** Generated automatically
3. **NetAnim GUI:** Open it, load animation.xml, hit Play → watch routers light up and packets fly between them

Verification Plan

Build & Run

```
cd /mnt/c/Users/shrir/Downloads/ns-3.48  
./ns3 run "routing-sim --routers=20 --topology=random"
```

Verify animation.xml

```
ls -la animation.xml
```

View in NetAnim

```
~/netanim/build/netanim  
# Then: File → Open → animation.xml
```

Test all topologies

```
./ns3 run "routing-sim --topology=ring --routers=10"  
./ns3 run "routing-sim --topology=mesh --routers=8"  
./ns3 run "routing-sim --topology=tree --routers=15"  
./ns3 run "routing-sim --topology=random --routers=50"
```

Test scaling

```
./ns3 run "routing-sim --routers=100 --topology=random --packets=200"
```

Open Questions

[!IMPORTANT] **Node layout for NetAnim:** How should routers be arranged visually?

- **Circle layout** (default) — works for all topologies, looks clean
- **Grid layout** — good for mesh
- **Tree layout** — good for tree topology

I'll default to **circle layout** since it works universally. Good?

[!NOTE] **Random link delay:** Our simulator assigns random per-link latency (1-10ms). In NS-3, I can either:

- Use a **fixed delay** for all links (simpler, cleaner for benchmarking)
- Use **random delay per link** (matches our simulator)

I'll implement **both** — fixed by default, with a `--randomDelay` flag to enable per-link random delays.